

STAYBID ENGINEERING

Bid System

A to Z — complete reference

Version 200 · 2026-05-24

21 sections · Negotiate · Bid · Flash · Counter · Payment · Stay

CONFIDENTIAL — INTERNAL

StayBid — Complete Bid System A-to-Z (v200, 2026-05-24)

Yeh document StayBid ke bid system ka **complete har step**, har rule, har logic, har link, har section, har button, har file path, har database column, har timing rule — sab kuch step-by-step Hinglish mein hai. Customer bid kaise place karta hai se lekar payment, check-in, check-out tak full lifecycle.

Section A — Yeh kya hai? (System overview)

StayBid ek **reverse-auction hotel platform** hai. Customer hotel ke MRP/live rate par book karne ke bajaye **apni price quote** kar sakta hai. Hotel partner us bid ko **accept / counter / reject** karta hai. Accept hone ke baad customer **15 minute ke andar** pay karke booking confirm kare warna bid auto-expire.

Bid 4 alag flows se launch hota hai:

Flow	Surface	Hotels per bid	Timer	Use case
Negotiate	<code>/hotels/[id]</code> Negotiate modal	1 (single hotel)	3 hours	Specific hotel pasand hai, price tweak karna hai
Simple Bid	<code>/hotels/[id]</code> Bid button	1 (single hotel)	3 hours	Quick single-hotel offer
Reverse Auction	<code>/bid</code> page	N hotels in 1 city	1 hour	Best price grab — broadcast karo, jo accept kare jeet jaye
Flash Deal Book	Flash story / <code>/flash-deals</code>	1 (instant book)	—	Pre-priced flash deal — bid nahi, direct payment

Section B — `/bid` reverse-auction (3-step flow)

File: `app/bid/page.tsx`

Step 1 — Where & When (Destination + dates)

- 3-up grid of supported cities (Mussoorie / Dhanaulti / Rishikesh / Dehradun / Shimla / Manali)
- Check-in + check-out date pickers (default: tomorrow + day-after)
- Validation: check-out > check-in, both ≤ 365 days ahead

Step 2 — Your Stay (Guests + room config + add-ons)

- Adults / Children / Kids in one row
- Rooms count
- Meal Plan 4 tiles per row (CP/MAP/AP/EP — Breakfast / Half board / Full board / Room only)

- Occasion chip row (Honeymoon / Anniversary / Birthday / Business / Family)
- Add-ons chip row (Airport pickup / Spa / Late checkout / Bonfire)

Step 3 — Your Price (Budget + tier preset + auction launch)

- **3 preset buttons:**
- 🏆 **Premium** → targets 4-5★ hotels, price preset = above-market
- ✨ **Smart** → targets all hotels, price preset = market level
- 💰 **Budget** → targets ≤4★ hotels, price preset = below-market
- **Slider** for fine price tuning — snaps to ₹100 multiples (`lib/price-snap.ts`)
- **v164 lowest-price guarantee:** computed deal = $\max(\text{slider_value}, \text{livePrice} * 0.92)$ — i.e. NEVER above (livePrice – 8%). LivePrice itself is already below OTAs (Section O — Pricing Spine).
- **Launch button** → fires the broadcast loop

Auction launch loop

1. POST `/api/bids/request` — creates ONE `bid_requests` row with `requestId`, dates, guests
2. For each target hotel (filtered by star tier):
 - POST `/api/bids/place` with `{ hotelId, roomId, amount, requestId, flow: "place" }`
 - All N bids share the same `requestId` → that's the same logical broadcast
3. Each hotel gets a PENDING bid with `expiresAt = now + 1h`

Celebration success screen (v163)

- Confetti animation
- Burst badge ("Bid Launched!")
- CountUp counter for "N hotels notified"
- **Live auction panel** — every PENDING bid streams in with status + countdown timer; when a hotel accepts, the customer sees the green pill on THIS screen, no need to jump to `/my-bids`

v194 "Pay Now & Grab" CTA

- When a bid auto-accepts on this same success screen, a green CTA appears
- Click → `router.push("/my-bids?payNow=<bidId>")`
- `/my-bids` reads `?payNow=<id>` query param on mount → auto-opens `BookingReview` modal for that bid → direct to Razorpay payment

Section C — `/hotels/[id]` Negotiate + simple Bid + "Your offers"

File: `app/hotels/[id]/page.tsx`

Negotiate modal flow

Open trigger: "Negotiate Price" button on hotel detail page.

Pre-flight gate (v200 `interceptIfActiveBidHere`):

```
if (pageActiveBids.length > 0) {
  alert("You already have an active bid on this hotel...");
  router.push(`/my-bids#bid-${bid.id}`);
  return; // modal never opens
}
```

Modal anatomy:

-  LIVE pulsing pill + "⚡ AI BIDDING ARENA" header
- **Probability ring** (animated SVG, red → green color shift)
- **Slot-machine number** with gold-shimmer animation
- **Rainbow slider** (red → amber → green gradient, gold thumb)
- **3 quick-pick chips**:
 - 💰 Max Saving (82% of floor)
 - ★ Smart Bid (90% of floor)
 - ⚡ Instant Book (100% of floor)
- 📺 **LIVE AI ticker** — rotates 4 tips every 12s
- 📊 **Sparkline** — 14-day demand bars
- 💎 **StayPoints win-teaser** + floating particles in instant-confirm zone

Two branches based on amount vs floor:

Branch	Condition	Backend
Above-floor	<code>amount >= floorPrice</code>	Razorpay payment FIRST → bid scheduled with <code>auto_accept_at</code> (tier-based delay)
Below-floor	<code>amount < floorPrice</code>	Submitted at <code>floorPrice</code> (DB constraint) + message token "Guest's preferred price: ₹X/night" preserves intent. NO payment yet. Hotel reviews + counters.

`executeNegotiate()` — the above-floor path

1. POST `/api/bids/request` → creates `bid_requests` row
2. Razorpay order created via `/api/razorpay/order`
3. Razorpay modal opens, user pays
4. `/api/razorpay/verify` HMAC check
5. POST `/api/bids/place` → bid inserted as PENDING with `expiresAt = now + 3h`
6. POST `/api/bids/[id]/schedule-accept` → records `auto_accept_at` (now + bidder_tier_delay) + `bidder_tier`
7. Customer redirected to `/my-bids` → sees `AutoAcceptCountdown` chip
8. Two paths to acceptance:
 - Customer watches: timer hits 0 → POST `/api/bids/[id]/trigger-accept` flips to ACCEPTED + sends email
 - Customer leaves: cron `/api/cron/expire-holds` calls RPC `mark_expired_holds()` which runs `auto_accept_eligible_bids()` → flips PENDING → ACCEPTED at scheduled time

Simple Bid (`handleBid`)

- Single-hotel quick offer (no slider, just slider preset)
- Same backend path as Negotiate but skips the modal — straight to PG

"Your offers" section

Visible only when customer has bids on THIS hotel. Renders per status:

Status	Card	Available actions
PENDING	Yellow border + countdown to <code>expiresAt</code>	💡 Update Budget (<code>UpdateBudgetInline</code>)
COUNTER	Champagne border + hotel's counter amount	✓ Accept / ✕ Decline / 💡 Update Budget
ACCEPTED (unpaid)	Green border + 15-min payment timer	💳 Pay Now (opens <code>BookingReview</code>)
ACCEPTED (paid)	Confirmed pill + check-in dates locked	View Booking → <code>/bookings</code>

v198 Upgrade chip (ACCEPTED accepted-card)

- Shows breakdown: ₹X accepted + ₹Y extra = ₹Z/night
- Opens `UpgradeRoomModal` → swap to a different room in same hotel + pay delta
- POST `/api/bids/[id]/upgrade-room` validates same hotel + delta payment

v199 UpdateBudgetInline (shared component)

- File: `components/UpdateBudgetInline.tsx`
- Slider 0.5× → 2× of current amount, snaps to ₹100
- PATCHes `/api/bids/[id]/budget`
- The route also fans out across sibling bids of `/bid` broadcast (same `requestId`)
- Auto-flips to ACCEPTED if new amount ≥ `floorPrice`

Section D — `/api/bids/place` — The 409 conflict-check route

File: `app/api/bids/place/route.ts` (the file you just read)

Pre-flight checks (in order)

1. **Auth** — `authUserId(req)` from JWT
2. **Required fields** — `hotelId`, `roomId`, `amount`
3. **Floor-price check** (skipped when `dealId` present):
 - Read `rooms.floorPrice`
 - If `amount < floor` → return `400 { error: "Amount too low. Minimum: ₹{floor}" }`
4. **v200 ONE-BID-PER-HOTEL guard** (`findActiveBidOnHotel`):
 - Query: `bids?customerId=in.(...)&hotelId=eq.<id>&status=in.(PENDING,COUNTER,ACCEPTED)`
 - **Drops** `expiresAt > now()` filter (v195 had a bug — stale ACCEPTED-unpaid bids past 15-min window had `expiresAt` in past → silently dropped → customer placed duplicates → 12 stacked ACCEPTED bills)
 - ACCEPTED rows only lock IF no paid booking yet (check `bookings.paidAmount > 0`)
 - Exempts same-requestId rows (so `/bid` broadcast across N hotels doesn't 409 itself)
 - On conflict → `409 { error: "You already have an active bid on <hotel>...", conflict: { bidId, hotelName, status, amount, ... } }`

Insert path (after pre-flight passes)

1. **v196 auto-accept rule** (only for `flow === "place"`, the `/bid` reverse auction):
 - Re-read floor (not from earlier check — `dealId`-bypass bids never auto-accept)
 - If `floor > 0 && amount >= floor` → `autoAccept = true`
2. **Insert into `bids`**:
 - `status: autoAccept ? "ACCEPTED" : "PENDING"`
 - `expiresAt: autoAccept ? now + 15min : (flow==="place" ? now+1h : now+3h)`
 - `isBestDeal: false`
3. **Return** `{ bid, autoAccepted }`

v200 per-flow timer

```
const NEGOTIATE_MS = 3 * 3600_000; // /hotels page Negotiate + simple Bid
const PLACE_MS      = 1 * 3600_000; // /bid reverse auction
const expiresAtFor = (flow) => flow === "place" ? PLACE_MS : NEGOTIATE_MS;
```

Section E — Bid lifecycle states (full state machine)

Status	Set by	Means	Next states
PENDING	/api/bids/place	Customer placed, hotel hasn't acted	ACCEPTED / COUNTER / REJECTED / EXPIRED
COUNTER	Partner Bid Inbox	Hotel counter-offered	ACCEPTED (customer accept) / REJECTED (customer decline) / PENDING (customer update budget)
ACCEPTED	Partner accept / customer counter-accept / cron auto-accept	Booking pending payment	CONFIRMED (paid) / EXPIRED (15-min window passed)
CONFIRMED	After Razorpay verify	Booking locked	CHECKED_IN
CHECKED_IN	Partner panel "Mark check-in"	Guest at hotel	CHECKED_OUT
CHECKED_OUT	Partner panel "Mark check-out"	Stay completed	(terminal)
REJECTED	Partner Bid Inbox	Hotel declined	(terminal)
EXPIRED	Cron mark_stale_pending_bids() / mark_expired_holds()	Timer ran out	(terminal)

Per-status expiry rules (read-time filter — lib/bid-expiry.ts)

Applied across /my-bids, partner Bid Inbox, /admin/bookings:

Status	Expiry rule
PAID ACCEPTED / CHECKED_IN / CHECKED_OUT	Never expire
PENDING with auto_accept_at	Expire 15 min after scheduled accept
PENDING without auto_accept_at	Expire at stamped expiresAt (1h /bid, 3h Negotiate). Fallback: 1h or 3h derived from message pattern
COUNTER	60 min after hotel posted counter
ACCEPTED & not paid	15 min after acceptance
REJECTED	30 min after decline
Hard IST midnight	ANY bid past next IST 00:00 after creation → stale regardless of status

v193 Server-side sweep (mid-2026-05-23)

RPC mark_stale_pending_bids() runs inside /api/cron/expire-holds every 15 min. Sweeps PENDING bids older than 6h to EXPIRED. 500-row cap per run.

Section F — Bidder tier system (v67 + v130)

File: lib/bidder-score.ts

Customer ka past bid history dekh ke 5-tier classification:

Tier	Avg <code>bid/floor</code> ratio	Auto-accept delay	Color	Label
PREMIUM	≥ 0.95	30 sec (instant)	green	"Premium Bidder"
STRONG	0.88 – 0.95	3 min (fast)	emerald	"Strong Bidder"
NORMAL	0.82 – 0.88	8 min (standard)	yellow	"Smart Bidder"
CAUTIOUS	0.75 – 0.82	20 min (careful)	amber	"Cautious Bidder"
LOWBALL	< 0.75	Manual hotel review only (<code>Infinity</code>)	red	"Lowball Pattern"
NEW	0 samples	8 min (default NORMAL)	blue	"New Bidder"

Score computation:

```
recent = last 10 bids
ratios = recent.map(b => b.amount / b.floorPrice).filter(...)
avg = mean(ratios)
tier = pickTier(avg)
```

Below-floor recovery: Customer's actual bid intent extracted from message token `"Guest's preferred price: ₹X/night"` so score uses real intent, not the floor-clamped DB value.

Customer-facing wording rule (v130): UI says "hotel will confirm" — **NEVER** "AI" / "auto" / "autopilot". Customer experience stays a clean partner story.

Section G — Hotel autopilot mode (v130 Hybrid Autopilot)

File: `lib/autopilot.ts` + `hotels.autopilot_mode` column

3 modes per hotel, set by partner in `/partner/dashboard` Profile tab:

Mode	Behavior
auto	Every tier-eligible bid auto-confirms at scheduled time
hybrid	Only PREMIUM + STRONG auto-confirm; NORMAL/CAUTIOUS wait for hotel review
manual	Every bid waits for hotel approve/reject

Critical: `/bid` reverse-auction broadcast deliberately STAYS manual — auto-accept on first acceptance would short-circuit competition. Only `/hotels/[id]` Negotiate + simple Bid respect autopilot mode.

LOWBALL exception: Even on `mode='auto'`, LOWBALL bidders ALWAYS wait for hotel review (`autoAcceptMs: Infinity`). v70 contract preserved.

Section H — `/api/bids/[id]/schedule-accept` (v70)

File: `app/api/bids/[id]/schedule-accept/route.ts`

POST endpoint that records the auto-accept schedule AFTER Razorpay verify.

Request body: `{ tier, autoAcceptMs }`

Side effects:

1. UPDATE `bids` SET `auto_accept_at = now() + autoAcceptMs`, `bidder_tier = tier`
2. INSERT `bid_acceptance_windows` row (tracks 15-min payment window state)

Why side-channel `/api/bids/auto-accept-info?ids=<id>` ?

Railway/Prisma `/api/bids/my` doesn't include new columns (`auto_accept_at` , `bidder_tier`) because Prisma client hasn't been regenerated. Frontend enriches the standard bid list with a separate Supabase fetch for these two fields. Zero Railway changes required.

Section I — `/api/bids/[id]/trigger-accept` (v70)

POST endpoint called by `AutoAcceptCountdown` when timer hits 0.

Idempotent: Validates `auto_accept_at <= now()` and `status === "PENDING"` before flipping. Repeated calls return `{ alreadyAccepted: true }`.

Side effects:

1. UPDATE `bids` SET `status = "ACCEPTED"`, `acceptedAt = now()`
2. Sends confirmation email via `/api/email/confirm` (v71 — async)
3. Starts 15-min acceptance window (already running via `bid_acceptance_windows`)

Section J — `/my-bids` (Customer bid history)

File: `app/my-bids/page.tsx`

Polling + normalization

- Polls `/api/bids/my` every 5 sec
- Normalizes dates from `bid_requests` join OR `localStorage.bid_dates_<bidId>` fallback
- Diffs status transitions to fire toast notifications

Filter pills

- All / Active / Accepted / Pending Payment / History
- `_isPlaceBid` derived via regex on message: `/\bGuest bid\b/i.test(msg) || /max ₹/i.test(msg)`
- Filtered via `filterActiveBids()` from `lib/bid-expiry.ts`

Countdowns

- PENDING with `auto_accept_at` → `<AutoAcceptCountdown>` ticking to scheduled time
- ACCEPTED unpaid → `<AcceptedBidTimer>` ticking to 15-min payment deadline
- COUNTER → 60-min countdown to decline-or-pay deadline

Counter-accept / counter-reject

- POST `/api/bids/[id]/counter-accept` → flips COUNTER → ACCEPTED at counterAmount, starts payment window
- POST `/api/bids/[id]/counter-reject` → flips COUNTER → REJECTED, terminal

Pay Now flow

- `<BookingReview>` modal opens (Section L)
- 3 payment paths: Pay Full / Hold 24h / Pay at Hotel

isPaid detection

`isBidPaid(b)` from `lib/bid-expiry.ts`:

```
const m = String(b?.message || "");
return m.includes("Razorpay:") || m.includes("razorpay_payment_id");
```

v194 `?payNow=<id>` deeplink

On mount, reads query param → finds matching bid → auto-opens BookingReview → clears param via `router.replace(pathname)`.

Section K — Hold lifecycle (v66 → v69)

24-hour hold option lets customer pay a small amount NOW to lock the price, settle balance later.

Hold tier defaults (by total booking amount)

Booking total	Hold amount
≤ ₹2,000	₹99
₹2,000 – ₹5,000	₹199
₹5,000 – ₹10,000	₹299
₹10,000 – ₹15,000	₹399
₹15,000+	₹499

Per-hotel override via `hotel_hold_config.tier_overrides` (admin `/admin/hold-config`).

Tables

- `bid_holds` — per-bid hold record (`status` ∈ active/completed/expired/cancelled)
- `bid_acceptance_windows` — per-bid 15-min payment timer state
- `hotel_hold_config` — global + per-hotel toggles

Cron sweep

RPC `mark_expired_holds()` runs every 5 min via `/api/cron/expire-holds`:

- Expires `bid_holds` past `expires_at`
- Expires `bid_acceptance_windows` past their 15-min window
- Auto-accepts eligible bids via `auto_accept_eligible_bids()` (LOWBALL skipped — `auto_accept_at` IS NULL)
- v193 → also calls `mark_stale_pending_bids()` for 6h+ PENDING sweep

HoldBanner on `/bookings`

Live HH:MM:SS countdown chip + "Pay Balance" CTA. 3 states:

- Active (gold, with countdown)
- Expired (red, dismissable)
- Pay-at-Hotel (gold info banner, no countdown — settled at desk)

Section L — BookingReview (v66 — 3 payment paths)

File: `components/BookingReview.tsx`

Every booking flow (Book Now / Flash Deal / Negotiate above-floor / Counter Accept / My Bids Pay-Now) lands here before Razorpay charges.

3 path cards:

1. 🌟 **Pay Full** — `total = nights × rate` — instant confirm
2. 🔒 **Hold for 24h** — `total = computeHoldAmount(bookingTotal)` — lock price for 24h
3. 🏨 **Pay Hold + Settle at Hotel** — `total = hold` — pay online, balance at desk

Trip details displayed:

- Hotel name + city
- Check-in / check-out dates (locked from bid)
- Number of nights, guests, room type
- Per-night rate × nights breakdown

Razorpay flow:

1. User picks path → `total` computed
2. POST `/api/razorpay/order` (v125 REST + v125.1 self-healing keys)
3. `openRazorpayCheckout({amount, ...})` opens modal
4. On success → POST `/api/razorpay/verify` (HMAC check)
5. POST `/api/bids/[id]/pay` writes `bid_paid_amounts` row + appends `Razorpay: pay_XXX` to bid message
6. If hold path: also POST `/api/holds` to write `bid_holds` row
7. Booking confirmed → redirect to `/bookings`

Section M — Partner Bid Inbox (`/partner/dashboard` Bid tab)

File: `app/partner/dashboard/page.tsx` + `app/api/partner/bids/route.ts`

Bid list

- Filter: All / Pending / Counter / Accepted / Rejected
- Sort: Newest first / Expiring soonest / Amount asc/desc
- Each card shows: customer phone (masked), bid amount, room type, dates, status pill, countdown, source badge (v94 — direct/creator/hotel-feed/flash)

Actions

Button	Endpoint	Body	Effect
✓ Accept	POST <code>/api/partner/bids/[id]</code>	<code>{ action: "accept" }</code>	Status → ACCEPTED, starts 15-min payment window
↻ Counter	POST <code>/api/partner/bids/[id]</code>	<code>{ action: "counter", counterAmount, addons }</code>	Status → COUNTER, customer has 60min to accept/decline
✕ Reject	POST <code>/api/partner/bids/[id]</code>	<code>{ action: "reject", reason? }</code>	Status → REJECTED, terminal

Counter add-on catalog (v129 — `lib/counter-addons.ts`)

Free-text "Message to Guest" textarea REMOVED (was leaking phone/email/WhatsApp through chat-free anti-bypass surface). Replaced with structured chip picker:

- Free breakfast
- Late checkout
- Room upgrade
- Welcome drink
- Spa discount
- Airport transfer
- Etc.

Customer sees them as pills in `/my-bids` counter card (parsed via `parseAddons()`).

Section N — Cron jobs (full schedule)

Vercel cron (Hobby 2-cap, both filled):

- `/api/cron/pricing` daily 4:00 AM — full OTA scrape + recalc + flash drop
- `/api/cron/lifecycle` daily 4:05 AM — daily bid lifecycle report

cron-job.org (the rest):

| Cron | Schedule | Purpose |

|---|---|---|

| `/api/cron/expire-holds` | every 15 min | RPC `mark_expired_holds()` — holds + windows + auto-accept + v193 stale-PENDING sweep |

| `/api/cron/flash-drop` | every 15 min | Fast room recalc + flash deal drops |

| `/api/cron/feedback-lifecycle` | hourly | 5 sweeps including hotel scorecard refresh |

| `/api/cron/price-spine` | hourly | Recompute `room_date_price` table for next 75 days |

| `/api/cron/auto-approve-content` | hourly | Sweep `PENDING_HOTEL_APPROVAL > 24h` → `AUTO_APPROVED` |

| `/api/cron/post-stay-nudge` | daily | Inspiration banner trigger 24-48h after checkout |

| `/api/cron/view-milestone-rewards` | daily | ₹50 at 1k views + ₹200 at 10k views |

| `/api/cron/creator-upgrade-eval` | weekly | Type A auto-promote + Type B admin-review eval |

| `/api/cron/support-auto-resolve` | daily | Auto-resolve stale support tickets |

Auth: `?token=<CRON_SECRET | "staybid-cron-dev">` query param.

Section O — Pricing Spine (v165 → v168)

Files: `lib/pricing/spine.ts` + `lib/pricing/read-spine.ts` + `room_date_price` table

The platform's price now has **ONE** source of truth.

Why this exists

Before v165: TWO disconnected engines (`lib/ai-pricing.ts` demand model vs `lib/pricing/engine.ts` competitor model) — hotel page / `/bid` / flash could each show a DIFFERENT number for the same room.

The spine table

`room_date_price` — one row per (room × date):

- `base_rate` — MRP
- `live_price` — what customer sees on hotel page + room cards
- `bid_floor` — minimum acceptable bid amount

- `flash_price` — discounted price for flash deal
- `vacancy` — units available
- `demand_score` — 0-100
- `competitor_min` — cheapest OTA price (scraped via OTA scraper)
- `factors` — JSONB explaining the math

Lowest-price guarantee (baked in)

`live_price` is ALWAYS below `competitor_min` (cheaper than OTAs). `bid_floor` is ALWAYS below `live_price`. So:

- Hotel-page live rate < every OTA
- `/bid` deal price < live rate (v164 enforces ≥8% below)
- Flash deal price < bid floor

`resolveSpinePrices()` — single accessor

```
import { resolveSpinePrices } from "@lib/pricing/read-spine";
const prices = await resolveSpinePrices({ roomId, fromDate, toDate });
// → { live_price, bid_floor, flash_price, ... }
```

Reads cache → if miss, computes on-the-fly via `lib/pricing/spine.ts`. Never breaks (falls back to local compute if spine unreachable).

Hotel-page room cards

60s recalc effect overrides only `.price` field with `live_price` from spine. Demand/trend badges stay local.

`/bid` page

Reads spine to compute the auction deal price + targets star tier filter.

Synthetic flash deals (v168)

`/api/flash/near` synthesizes flash cards from `flash_price` when no real `flash_deals` row exists. Real `flash_deals` rows stay hotel/cron-managed.

Section P — `lib/paid-amount.ts` — display amount priority chain

Single source of truth for "what is the customer's bid amount" across `/my-bids`, `/bookings`, hotel page.

`resolveBidDisplayAmount()` priority

1. **counterAmount** (if hotel countered AND status !== PENDING) — that's the active price now
2. **Customer's preferred price from message** (below-floor recovery via `"preferred price: ₹X"` regex)
3. **Server-side paidPerNight** (from `bid_paid_amounts`)
4. **bid.amount** (DB value, may be floor for below-floor cases)

`resolvePaidAmount()` — for total paid

1. server-side `bid_paid_amounts.paidTotal` (via `/api/bid/paid?ids=...`)
2. `paid:X` token in `bid.message` (legacy)
3. `localStorage.paid_amount_<bidId>` (same-device only)
4. `localStorage.deal_price_<bidId>` (legacy per-night)

5. `bid.totalAmount` || `bid.amount`

Section Q — Booking-source attribution (v94)

File: `app/api/attribution/record/route.ts` + `bid_attributions` table

Every booking carries source channel end-to-end. Visible in creator hub, partner panel, admin panel.

Source	When	Commission?
<code>direct</code>	URL typed / SEO	No
<code>creator</code>	Reel from CREATOR user → Book/Bid	Yes (slab-based v95)
<code>hotel-feed</code>	Reel from HOTEL user → Book/Bid	No (it's hotel's own marketing)
<code>flash</code>	Flash deal story → direct book	No

Capture path

1. Customer taps reel CTA → URL gets `?src=creator&cid=<users.id>&via=<handle>&ctype=CREATOR&vid=<social_posts.id>`
2. Hotel page reads params, persists to `localStorage.sb_attribution_<hotelId>` (24h TTL)
3. On bid success → `recordAttribution({ bidId, hotelId, paidTotal, flow, attribution })`
4. Writes `bid_attributions` row + auto-inserts `influencer_commissions` row for creator-attributed paid bookings

Commission engine (v95)

4 slabs by monthly attributed booking count:

- 1–25 → 5%
- 26–50 → 7%
- 51–100 → 10%
- 101–300 → 12%

Plus loyalty bonus: +1% at 3 consecutive months, +2% at 6 months. Audit trail in `bid_attributions.metadata.commission` is IMMUTABLE.

Section R — Active bid conflict (409 handling)

File: `components/ActiveBidConflictSheet.tsx`

When `/api/bids/place` returns 409 with `{conflict: {...}}`, the conflict sheet opens:

- Hotel name + city
- Current bid amount + status
- "Open in My Bids" CTA → `/my-bids#bid-<id>`
- Inline `<UpdateBudgetInline openByDefault={true}>` — change the bid right there without going to `/my-bids`

Section S — v195 → v200 evolution (recent)

Version	What
v195	One-bid-per-CITY rule introduced (initial implementation)
v196	<code>/bid</code> auto-accept rule — hotels with <code>floor ≤ offer</code> auto-confirm INSTANTLY (Sachin product decision)
v197	"Accepted at ₹X" static card on <code>/hotels/[id]</code> for accepted bid; other rooms still showed Book/Negotiate (incomplete fix)
v198	Pre-fill check-in/out from accepted bid + lock picker; upgrade chip with breakdown; <code>/api/bids/[id]/upgrade-room</code> for delta payment
v199	Shared <code>UpdateBudgetInline</code> component — mounted on <code>/my-bids</code> AND <code>/hotels/[id]</code> "Your offers" PENDING/COUNTER cards
v200	One-bid-per-HOTEL rule (CRITICAL FIX) — <code>findActiveBidInCity</code> renamed to <code>findActiveBidOnHotel</code> . Dropped <code>expiresAt > now()</code> filter (was silently allowing stale ACCEPTED-unpaid duplicates). Defense-in-depth <code>interceptIfActiveBidHere()</code> guard. Bottom sticky chip hidden when active bids exist. DB cleanup: 12 stacked ACCEPTED bids cleared via supervised SQL.

Outstanding v192–v194 (sitting on `main`, ready to merge)

- **v192 PR #117** — Widen upgrade-chip rule to PENDING/COUNTER (not just ACCEPTED)
- **v193 PR #118** — RPC `mark_stale_pending_bids()` + cron wiring (already applied to Supabase — 93/120 stuck bids cleared on first call)
- **v194 PR #119** — `/bid` success "Pay Now & Grab" CTA → `?payNow=<id>` → `/my-bids` auto-opens `BookingReview`

Section T — How a complete bid plays out (worked example)

Scenario: Customer wants to bid ₹3500 on a 4★ Mussoorie hotel with `floorPrice = ₹3000`.

T1. Open hotel page

- `/hotels/mus01` loads
- Spine returns: `live_price=₹3800`, `bid_floor=₹3000`
- No existing bids → "Negotiate Price" button enabled

T2. Open Negotiate modal

- Slider defaults to 90% of floor (₹2700) → labeled "Smart Bid"
- Customer drags to ₹3500 → above floor → "⚡ Instant Confirm" green pill
- Probability ring shows 95%
- AI ticker says "Hotels with this autopilot_mode='auto' confirm in ~3 min"

T3. Submit (above-floor path)

1. POST `/api/bids/request` → `{ requestId: "req_abc", checkIn, checkOut }`
2. POST `/api/razorpay/order { amount: 350000 }` → Razorpay order created
3. Customer enters card details → pays ₹3500
4. POST `/api/razorpay/verify` → HMAC verified

5. POST `/api/bids/place` { hotelId: "mus01", roomId, amount: 3500, requestId, flow: "negotiate" }:
- Pre-flight: no conflict
 - Floor check: $3500 \geq 3000$ ✓
 - Insert: status=PENDING, expiresAt=now+3h, isBestDeal=false
6. POST `/api/bids/[id]/schedule-accept`:
- Compute bidder_tier = STRONG (based on customer's history)
 - autoAcceptMs = 180_000 (3 min)
 - UPDATE bids SET auto_accept_at = now + 3min, bidder_tier = "STRONG"
 - INSERT bid_acceptance_windows row

T4. Customer redirected to `/my-bids`

- Poll picks up the new bid
- `<AutoAcceptCountdown>` renders: "Hotel confirming in 02:58..."
- Status pill: PENDING (yellow)

T5. 3 minutes pass

- Timer hits 0 → component fires POST `/api/bids/[id]/trigger-accept`
- Route validates `auto_accept_at <= now` and `status === "PENDING"` ✓
- UPDATE bids SET status="ACCEPTED", acceptedAt=now
- Email sent via `/api/email/confirm`
- 15-min payment window starts

T6. Customer pays balance

- `<AcceptedBidTimer>` renders: "Pay within 14:32 to confirm"
- Customer taps "Pay Now" → `<BookingReview>` opens
- 3 paths shown (Pay Full ₹3500 / Hold ₹199 / Hold + Settle at Hotel ₹199)
- Picks Pay Full → Razorpay flow
- POST `/api/bids/[id]/pay` writes `bid_paid_amounts` + appends `Razorpay: pay_XXX` to message

T7. Booking confirmed

- Status now ACCEPTED + paid → won't expire (`isBidPaid()` returns true)
- Shows in `/bookings` as upcoming
- Hotel partner sees in `/partner/dashboard` Bookings tab

T8. Check-in day

- Partner taps "Mark Check-in" → status → CHECKED_IN
- BookingChat opens between customer + hotel (v71 — anti-bypass sanitizer active)

T9. Check-out day

- Partner taps "Mark Check-out" → status → CHECKED_OUT
 - Customer gets stay-feedback prompt (v127 — 5 smileys)
 - 12h window to submit (v127.2)
 - StayPoints credited (5 per ₹100 spent)
 - Hotel scorecard refreshes within 6h (Sweep 5 of feedback-lifecycle cron)
-

Section U — Anti-patterns (what NEVER to do)

Bid system has 200 versions of accumulated discipline. The most load-bearing rules:

- **NEVER** edit `bid.amount` in DB to "fix" below-floor mismatch. Real intent is in message token; `bid.amount` is floor on purpose. Use `resolveBidDisplayAmount` everywhere.
- **NEVER** restore the instant `api.acceptBid` call in `executeNegotiate` above-floor path. The v70 system depends on PENDING with `auto_accept_at` scheduled.
- **NEVER** drop the `auto_accept_at IS NOT NULL` guard from `auto_accept_eligible_bids()` RPC — that's what makes LOWBALL bids require manual hotel review.
- **NEVER** add chat to PENDING bids. The v25 anti-bypass rule gates chat behind ACCEPTED status.
- **NEVER** show "AI" / "auto" wording in customer-facing copy on bid lifecycle. Customer sees "hotel confirming" — never the autopilot mechanism.
- **NEVER** let `/bid` reverse-auction respect `autopilot_mode`. It STAYS manual — auto-accept would short-circuit competition.
- **NEVER** add price input without `snap100()` from `lib/price-snap.ts`. ₹100 multiples are platform-wide.
- **NEVER** widen the v164 lowest-price guarantee. Auction deal MUST be ≥8% below `livePrice`; `livePrice` is below OTAs.
- **NEVER** restore the `expiresAt > now()` filter in `findActiveBidOnHotel`. That was the v195 bug allowing 12 stacked ACCEPTED duplicates.
- **NEVER** drop the same-`requestId` exemption in conflict-check. `/bid` broadcast across N hotels uses ONE `requestId`; without exemption it would 409 itself.
- **NEVER** point `/api/cron/auto-approve-content` at `PENDING_ADMIN_REVIEW`. Those must wait for human admin (v160 contract).
- **NEVER** raise `mark_stale_pending_bids()` row cap past ~2000 without checking PostgREST timeout.
- **NEVER** lower the 6h threshold in `mark_stale_pending_bids()` without also lowering client rule in `lib/bid-expiry.ts:106`. Diverging values create visible "stuck" gap.

TL;DR — bid lifecycle in 1 sentence

Customer places bid → (above-floor: Razorpay pays first + auto-accept scheduled at tier-based delay | below-floor: bid submitted at floor, hotel manually accepts/counters/rejects) → on ACCEPTED, 15-min payment window opens → on payment, status → CONFIRMED → check-in marks CHECKED_IN → checkout marks CHECKED_OUT → terminal. Stale rows swept by `mark_expired_holds()` + `mark_stale_pending_bids()` cons.

— End of document —